MINI PROJECT REPORT

EV CHARGING STATION PLACEMENT USING LINEAR REGRESSION, HILL CLIMBING AND SIMULATED ANNEALING

Submitted by:

Easala Jagadeesh

2411021061183

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Under the guidance of

Sharutha J

Associate Professor, ASAC

Alliance School of Advanced Computing

Alliance University, Bengaluru

October-2026

COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on **AI/ML Foundations** is submitted by **Easala Jagadeesh** [Uni. Reg. No. 2411021061183] in partial fulfilment for the award of the degree of Bachelor of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026-27. This report embodies the results of original work and studies conducted by the student and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

- - - - - - - - - - - - - - - - - - - -

Sharutha J

Associate Professor



COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled **EV CHARGING STATION PLACEMENT USING LINEAR REGRESSION, HILL CLIMBING AND SIMULATED ANNEALING**, submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (CSE) at Alliance University, is a record of my original work carried out under the supervision of Sharutha J, Associate Professor.

I further confirm that all views, findings, and conclusions expressed herein are the result of sincere effort and academic inquiry, and have been reviewed and deliberated in consultation with the assigned mentor.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

Easala Jagadeesh	**2411021061183**	
Name of the Student	University Registration Number	Signature

Acknowledgement

I would like to express my heartfelt gratitude to everyone who contributed to the successful completion of this project titled **EV CHARGING STATION PLACEMENT USING LINEAR REGRESSION, HILL CLIMBING AND SIMULATED ANNEALING**.

First and foremost, I extend my sincere thanks to my mentor and guide, Sharutha J, for the unwavering support, insightful feedback, and encouragement throughout the development of this system. The guidance received in the subject AI/ML Foundations provided the perfect foundation for this work.

I am deeply grateful to my faculty members and peers whose guidance and technical inputs helped refine the project. Their collaborative spirit and constructive suggestions were instrumental in shaping the final outcome.

This project allowed me to formulate a real city-planning problem as a state space, compare two local search algorithms on the same map, and train a regression model from survey data. It strengthened my belief in the power of artificial intelligence to solve everyday problems in a simple and explainable way.

Lastly, I would like to thank everyone who supported me directly or indirectly during this journey. Your encouragement and belief in my work made this accomplishment possible.

Thank you.

Easala Jagadeesh

TABLE OF CONTENTS

Acknowledgement **4**

Abstract **6**

Chapter I- Introduction **7**

Problem Statement 7

Theory 7

Chapter II - System Configuration **10**

Algorithm 10

Source Code 11

Chapter III - Results **13**

Demand prediction 13

Station placement 13

Chapter IV - Conclusion **16**

Abstract

As electric vehicles become common, a city must decide where to build its few fast-charging stations. A station placed in a quiet area is wasted, while busy areas are left without charging. Two difficulties make the decision hard: charging demand has been measured in only a small part of the city, and the number of possible placements is very large. This report presents an AI-based planner that solves both problems with a simple machine-learning model and classical optimization algorithms.

The city is modelled as a 10 × 10 grid of 100 zones. Demand was surveyed in 25 zones, of which 24 remained after data cleaning. A simple linear regression model trained by ordinary least squares learned demand = 3.29 + 0.41 × cars and predicted the unseen test zones with a mean absolute error of 0.98 charging sessions per day. Station placement is formulated as a state-space optimization problem with 161,700 possible plans for three stations, and searched with steepest-ascent hill climbing and simulated annealing.

Over 40 independent runs simulated annealing found the best possible plan (394.4 sessions per day, confirmed by checking every plan) 38 times, while hill climbing found it only 1 time because it stops at local maxima. The report concludes that a simple learned model combined with simulated annealing gives fast, explainable and reliable infrastructure plans.

Chapter I- Introduction

Artificial Intelligence (AI) is the study of agents that perceive their environment and act to achieve goals. Many practical AI tasks can be described as finding the best state among a huge number of possible states, which is the job of optimization algorithms. Machine Learning (ML), a branch of AI, lets a program make predictions by learning patterns from data instead of following fixed rules.

Planning electric-vehicle (EV) charging infrastructure needs both ideas. First, the planner must know how much charging demand each part of the city has, but demand is only measured in a few surveyed zones, so it must be predicted for the rest from data. Second, the planner must choose the station locations that serve the most demand, and the number of possible choices is far too large to check by hand.

Problem Statement

A city divided into a 10 × 10 grid of 100 zones can afford only three fast-charging stations. A station serves its own zone and the four zones next to it, and a zone served by two stations is counted only once. Charging demand was measured in only 25 zones. The problems addressed are:

- to clean the survey data and learn a model that predicts the daily charging demand of every zone from the number of electric vehicles registered there;
- to formulate station placement as a state-space optimization problem;
- to search the 161,700 possible placements efficiently with hill climbing and simulated annealing; and
- to compare how reliably each algorithm finds the best plan, using an exhaustive check as the ground truth.

Theory

Simple linear regression

Simple linear regression models the relationship between one input x and one output y with a straight line $\hat{y} = a + b\,x$. Here x is the number of EVs registered in a zone and y is its daily charging demand (sessions per day). The parameters are chosen to minimise the sum of squared errors, which gives the closed-form ordinary least squares (OLS) solution

$$b = \Sigma\,(x_i - \bar{x})(y_i - \bar{y}) \,/\, \Sigma\,(x_i - \bar{x})^2,\quad a = \bar{y} - b\,\bar{x}$$

The slope b has a direct meaning: the number of extra charging sessions expected for every extra EV. The model is trained on part of the data and judged on unseen test rows with the mean absolute error, $MAE = (1/n)\,\Sigma\,|y_i - \hat{y}_i|$, measured in sessions per day.

Problem formulation as a state space

Component	Definition in the planner
State	the positions of the three stations, e.g. ((2, 2), (2, 8), (7, 6))
Initial state	three randomly chosen, different zones
Actions (moves)	move one station one zone up, down, left or right
Objective	total predicted demand of all covered zones (each zone counted once)
Goal	the state with the highest objective; there is no goal test, so this is optimization
Size	C(100, 3) = 161,700 possible placements

Table 1: Problem formulation

Steepest-ascent hill climbing

Hill climbing starts from a random state and repeatedly moves to the best neighbouring state. It stops when no neighbour is better. It is fast and uses very little memory, but the state where it stops is only a *local* maximum: every single move makes things worse, even though a much better plan may exist elsewhere. In this problem a typical trap is two stations crowded into the same busy area.

Simulated annealing

Simulated annealing also moves between neighbouring states, but it picks a random neighbour and sometimes accepts a *worse* one. A move that changes the score by Δ is always accepted if $\Delta > 0$ and otherwise accepted with probability

$$\mathbf{P(accept) = e^{\Delta / T}}$$

where the temperature T starts high, so bad moves are often accepted and the search can escape local maxima, and is multiplied by a cooling rate after every step, so the search gradually settles down. The planner uses $T_0 = 80$, a cooling rate of 0.9993 and 8,000 steps, and remembers the best state seen.

Property	Hill climbing	Simulated annealing
Next state	best neighbour	random neighbour
Accepts worse moves	never	sometimes, with probability e^(Δ/T)
Stops when	no neighbour is better (local maximum)	after a fixed number of steps
Escapes local maxima	no	yes, while the temperature is high
Cost	very low	higher (thousands of evaluations)

Table 2: Hill climbing compared with simulated annealing

Because three stations give 'only' 161,700 plans, a brute-force search that checks every plan is still possible here and is used as the ground truth to measure how often each algorithm finds the true best plan. For a real city with more zones and stations the number of plans grows far too quickly for brute force, which is why local search is needed.

Chapter II - System Configuration

EV Charge Planner is a Python application with a small web interface that runs in the browser. The regression, the optimization algorithms and the data preparation are written from scratch in plain Python without any AI or ML library. The implementation and all experiments were carried out on the following laptop.

Component	Specification
Laptop	Dell Inspiron 15 3525
Processor	AMD Ryzen 5 5625U with Radeon Graphics, 6 cores / 12 threads
Memory	8 GB RAM
Operating system	Windows 11 Home Single Language
Language	Python 3.12 (standard library only, no external libraries)
Web interface	HTML5, CSS3, JavaScript (ES6), served by Python's built-in http.server
Browser	Microsoft Edge (Chromium)
Version control	Git and GitHub

Table 3: System configuration

Algorithm

Data preparation and linear regression

1. Load the survey file (row, column, number of EVs, measured demand).
2. Drop every row whose demand value is missing.
3. Use the first 18 clean rows for training and the remaining rows for testing.
4. Compute the means of x and y, then the slope b and intercept a with the least-squares formulas.
5. Predict the test rows and compute the mean absolute error.
6. Apply the model to the EV count of all 100 zones to obtain the predicted demand map.

Coverage score

1. For every station, add its own zone and its four neighbours (inside the grid) to a set.
2. The score is the sum of the predicted demand of all zones in the set; using a set counts a zone served by two stations only once.

Hill climbing

1. Start from three random zones.
2. Generate every neighbouring state (one station moved one zone) and pick the one with the highest score.
3. If it is not better than the current state, stop and return the current state (local maximum).

4. Otherwise move to it and repeat from step 2.

Simulated annealing

1. Start from three random zones with temperature T = 80.
2. Pick a random neighbour and compute the change Δ in score.
3. Accept it if Δ > 0, otherwise accept it with probability e^(Δ / T).
4. Remember the best state seen, multiply T by 0.9993, and repeat for 8,000 steps.
5. Return the best state seen.

Source Code

The listings below are taken directly from the project file main.py.

Linear regression and error measure

```
def train_linear_regression(x, y):
    """
    Fits a simple linear regression model: y = a + b * x
    Uses standard Ordinary Least Squares (OLS) formulas:
      b = sum((x - mean_x) * (y - mean_y)) / sum((x - mean_x)^2)
      a = mean_y - b * mean_x
    """
    mean_x, mean_y = sum(x) / len(x), sum(y) / len(y)
    b = sum((xi - mean_x) * (yi - mean_y) for xi, yi in zip(x, y)) / sum((xi - mean_x) ** 2 for xi in x)
    a = mean_y - b * mean_x
    return a, b

def mean_absolute_error(actual, predicted):
    """Calculate Mean Absolute Error between actual and predicted values."""
    return sum(abs(p - q) for p, q in zip(actual, predicted)) / len(actual)
```

State space: coverage, score and neighbouring states

```
def covered_zones(state):
    """
    Returns the set of all zones covered by the stations.
    Each station covers its own zone plus its 4 orthogonal neighbors.
    Using a set prevents double-counting overlapping zones.
    """
    zones = set()
    for r, c in state:
        for dr, dc in [(0, 0), (1, 0), (-1, 0), (0, 1), (0, -1)]:
            if 0 <= r + dr < SIZE and 0 <= c + dc < SIZE:
                zones.add((r + dr, c + dc))
    return zones

def score(state, demand):
    """Calculate total demand served by the given station placement."""
    return sum(demand[z] for z in covered_zones(state))

def neighbours(state):
    """Generate all valid states reachable by moving one station by one step."""
    result = []
    for i, (r, c) in enumerate(state):
        for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            new = (r + dr, c + dc)
            if 0 <= new[0] < SIZE and 0 <= new[1] < SIZE and new not in state:
                result.append(state[:i] + (new,) + state[i + 1:])
    return result
```

Optimization algorithms

```
def hill_climbing(demand):
    """
    Steepest-Ascent Hill Climbing:
    Greedily moves to the neighboring state with the highest score.
```

```
    Stops when no neighbor has a better score (local maximum).
    """
    state = random_state()
    while True:
        best = max(neighbours(state), key=lambda s: score(s, demand))
        if score(best, demand) <= score(state, demand):
            return state
        state = best

def simulated_annealing(demand, temperature=80.0, cooling=0.9993, steps=8000, k=STATIONS):
    """
    Simulated Annealing:
    Stochastic local search. Always accepts better moves.
    Accepts worse moves with probability P = exp(delta / T).
    Temperature cools gradually: T = T * cooling_rate.
    This allows the algorithm to escape local maxima early in the search.
    """
    state = random_state(k)
    best = state
    for _ in range(steps):
        new = random.choice(neighbours(state))
        change = score(new, demand) - score(state, demand)
        if change > 0 or random.random() < math.exp(change / temperature):
            state = new
        if score(state, demand) > score(best, demand):
            best = state
        temperature *= cooling
    return best

def brute_force(demand):
    """
    Checks all C(100, 3) = 161,700 combinations to find the exact global maximum.
    Used as an exact baseline to verify heuristic algorithm performance.
    """
    all_zones = [(r, c) for r in range(SIZE) for c in range(SIZE)]
    return max(combinations(all_zones, STATIONS), key=lambda s: score(s, demand))
```

Chapter III - Results

Demand prediction

The survey contained 25 rows. One row had a missing demand value and was removed, leaving 24 rows, of which 18 were used for training and 6 for testing. The model learned

$$\textbf{demand} = 3.29 + 0.41 \times \textbf{cars}$$

which means that every extra electric vehicle in a zone adds about 0.41 charging sessions per day. On the 6 unseen test zones the mean absolute error was 0.98 sessions per day (Table 4). Applied to all 100 zones, the model predicts a total demand of 1338.4 sessions per day.

Zone (row, col)	EVs	Measured demand	Predicted demand	Error
(6, 5)	47	22.9	22.7	0.2
(6, 6)	64	29.2	29.7	0.5
(6, 9)	10	5.2	7.4	2.2
(9, 1)	10	8.8	7.4	1.4
(9, 6)	29	14.3	15.2	0.9
(9, 7)	20	10.9	11.5	0.6

Table 4: Predictions for the unseen test zones

Station placement

Both algorithms were run ten times with the program's fixed random seed, and every result was compared with the best possible plan found by checking all 161,700 placements.

Algorithm	Average score	Worst	Best	Found the best plan
Hill climbing	354.6	291.7	394.4	4 / 10
Simulated annealing	392.3	373.0	394.4	9 / 10
Brute force (exact)	394.4	-	-	always (checks every plan)

Table 5: Ten runs of each algorithm (score = charging sessions per day covered)

To measure reliability more precisely, each algorithm was then run 40 more times with a different seed. Simulated annealing found the best plan in 38 of 40 runs (95%), hill climbing in only 1 of 40 (2%) (Figure 1).

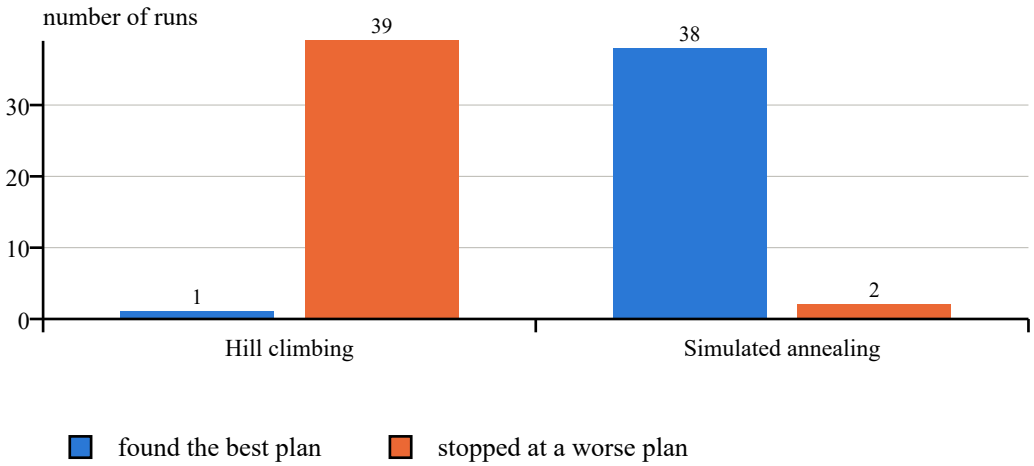


Figure 1: Outcome of 40 independent runs of each algorithm

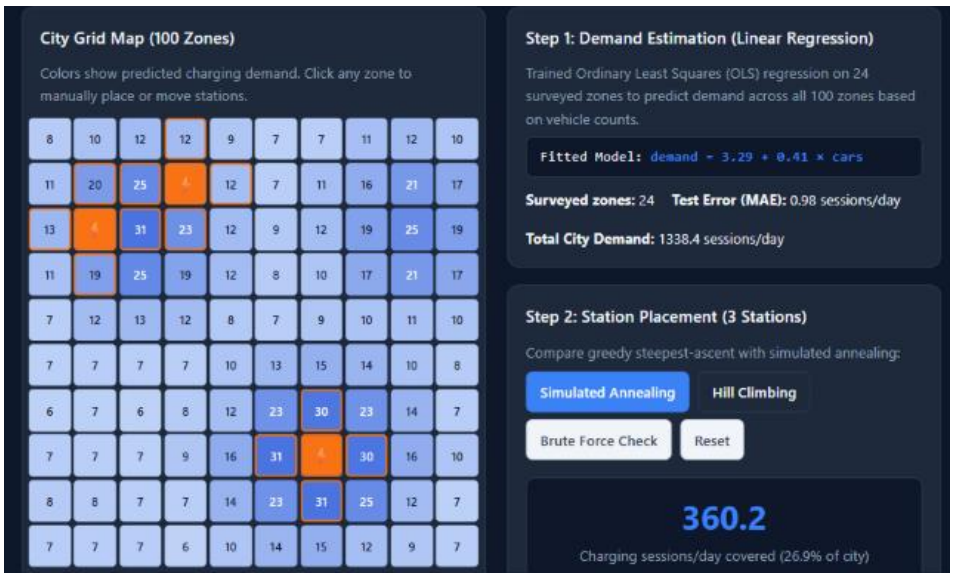


Figure 2: Hill climbing stuck at a local maximum: two stations crowded into the same top-left area (score 360.2)

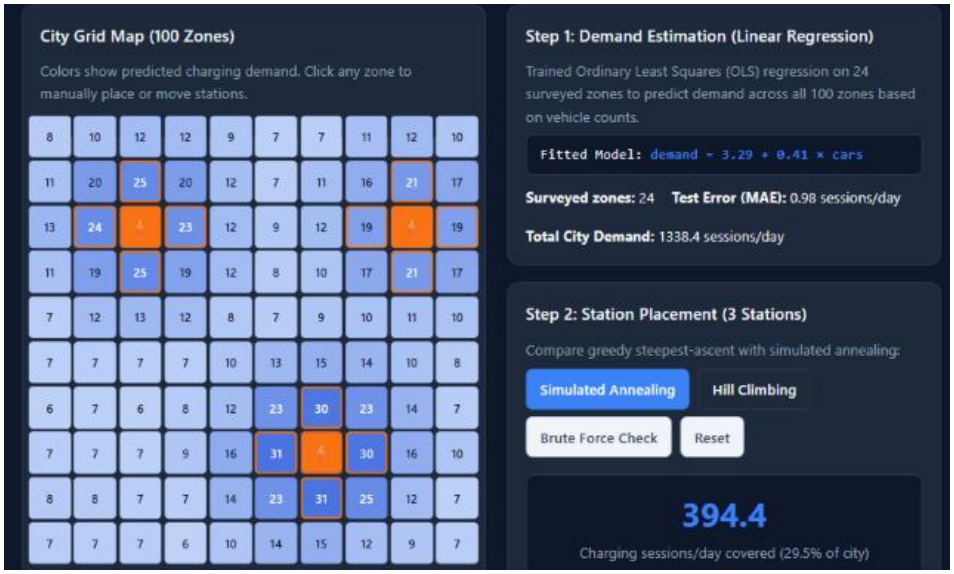


Figure 3: Simulated annealing's plan: one station in each of the city's three EV hotspots, covering 394.4 sessions per day (29.5% of the city)

The final plan places one station in each of the three areas with many electric vehicles, at (2, 2), (2, 8) and (7, 6). Hill climbing often ends with two stations next to each other in the same area: from there, moving either station one zone away loses more demand than it gains, so hill climbing stops, while simulated annealing's occasional downhill moves let it walk the extra station to the third hotspot.

Chapter IV - Conclusion

This report demonstrates that a simple machine-learning model and classical optimization can turn a partial survey into a sound infrastructure plan. Ordinary least squares learned the relationship between electric vehicles and charging demand from only 24 surveyed zones and predicted unseen zones with a mean error of 0.98 sessions per day.

Formulating placement as a state space made it possible to search 161,700 plans without checking them all. Hill climbing was fast but stopped at local maxima in most runs, while simulated annealing, by accepting some worse moves while the temperature was high, found the best plan in 38 of 40 runs. This illustrates the main lesson of local search: a little controlled randomness lets the search escape traps that a purely greedy method cannot.

Future work includes multiple linear regression with features such as population, offices and highway access, station capacities and building costs in the objective, a larger city where brute force is impossible, and random-restart hill climbing as a further comparison.